OPEN PROJECTS 2026 · AI / ML · TECHNICAL REPORT

Personalized Content Discovery *on the Netflix Prize*

Four recommenders, two objectives, one inconvenient truth: the model that predicts ratings best is not the model that ranks best.

TEAM rukdimaxx

MEMBERS Somil Agrawal · 23112099 S R Nivedhitha · 22411030

REPOSITORY

github.com/silversoul2213/ai-ml-cult

BOTTOM LINE

We built and benchmarked four recommenders on the Netflix Prize data. The stacked **Ensemble wins on rating accuracy (RMSE 0.905, MAE 0.706)**; the trivial bias **Baseline wins on ranking (MAP@10 0.184)**. These two goals genuinely diverge.

The proof is built into our own model: when the ensemble learned its blend weights to minimise error, it assigned the baseline — our best *ranker* — a weight of **exactly zero**. Accuracy and discovery are different products. Ship ranking from the baseline, ratings from the ensemble.

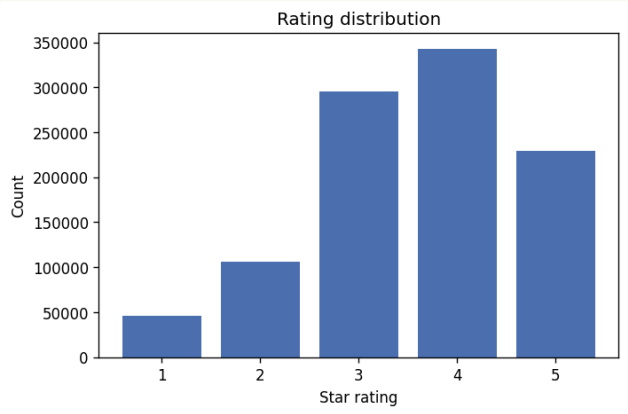
01 Problem Understanding

Streaming platforms live or die on content discovery. We learn a preference function $\hat{r}(u,i)$ that predicts how a user would rate an unseen title, then rank the catalogue to produce a personalised Top-K list. Two success axes are in play and they are **not the same: rating accuracy** (does the predicted star value match reality? — RMSE) and **ranking quality** (are the items we actually surface the ones the user will love? — MAP@10). We optimise and report both, and treat the gap between them as the central finding rather than a footnote — exactly how a real recommendation team is judged.

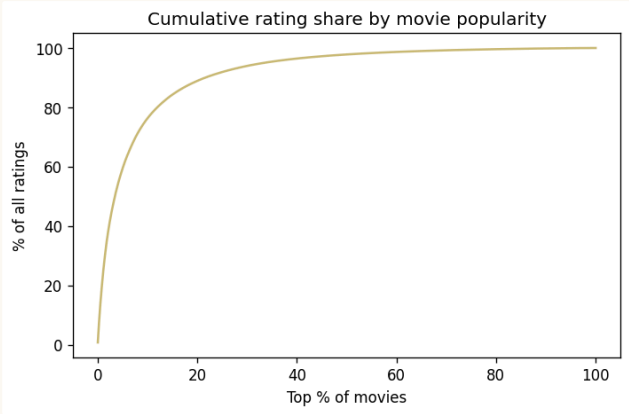
02 Exploratory Data Analysis

The full Netflix Prize corpus holds **100,480,507 ratings** from **480,189 users** across **17,770 movies** (1–5 stars, dated). Because training on 100M interactions on CPU buys nothing for a methodological comparison, we sample **20,000 users** and keep *every* rating they made in `combined_data_1` — sampling *users*, not rows, preserves real per-user density. After requiring ≥5 ratings per user, the working set is:

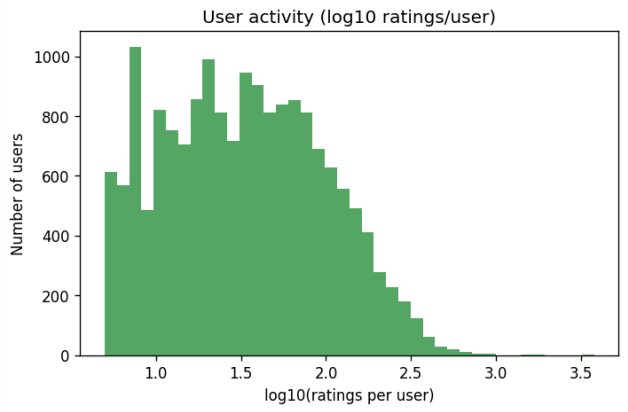
1.02M	17,244	4,496	98.7%
RATINGS	USERS	MOVIES	SPARSITY



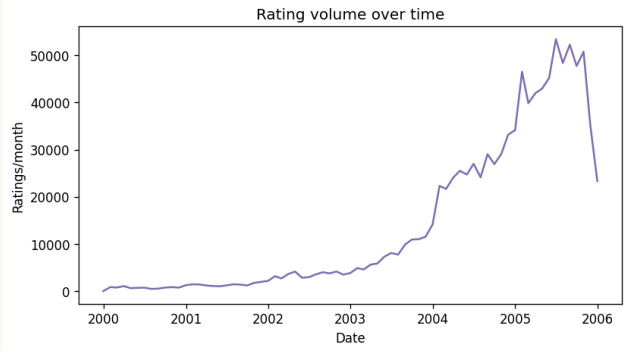
Ratings skew positive. Mean 3.59, median 4.0 — 4★ is the modal score; 1★ is rare. A near-mean predictor is already hard to beat on raw error.



A steep long tail. The top 11.97% of movies absorb 80% of all ratings — popularity bias is the enemy of genuine discovery.



Power-user skew. Ratings per user span 5–3,769 (median 32, mean 59) — a heavy tail of heavy raters that motivates per-user bias terms.



Volume grows over time. Recency matters, which is why our train/test split is temporal rather than random.

Implications. 98.7% sparsity rewards models that generalise across empty cells; the steep tail warns that a popularity-only system looks strong on head items yet fails at discovery; power-user skew makes bias correction non-negotiable.

03 Methodology

Train / test split. Per-user *temporal* hold-out: for every user we sort ratings by date and move the most recent 20% to test. This mimics deployment — predict the future from the past — and removes look-ahead leakage. A further 10% slice of the training set is reserved as *validation* for early-stopping and ensemble weighting.

Relevance. For ranking, an item is relevant iff the user's true held-out rating is ≥ 3.5.

Top-10 procedure. For each evaluated user we score a candidate pool of their true held-out items plus up to 200 sampled unseen negatives, sort by predicted rating, and take the top 10.

MAP@10. Average Precision@10 per user — rewarding relevant items ranked high — averaged over the ≤2,000 evaluated users who hold at least one relevant item. Full procedure in `src/evaluate.py`.

04 Model Design

Four approaches of increasing sophistication, all in pure NumPy / SciPy / scikit-learn — no black-box library — so every prediction is transparent and reproducible.

— **Baseline (bias).** $\hat{r} = \mu + b_u + b_i$ with shrunk bias estimates (item first, then user on the residual). Closed-form, fast, and a fierce reference.

— **Item-Item CF.** Adjusted-cosine similarity on mean-centred items, top-50 neighbours, with significance shrinkage and a minimum co-rating overlap to discount thin, unreliable similarities.

— **Matrix Factorization (SGD).** $\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$, 50 latent factors, L2-regularised SGD with validation early-stopping; predictions clipped to [1,5].

— **Ensemble (stacked).** A non-negative least-squares blend of the three base models, with weights learned on the held-out validation slice — the Netflix-Prize-winning idea, applied honestly out-of-fold.

05 Evaluation Metrics

RMSE (mandatory) certifies rating accuracy; **MAP@10** (mandatory, relevance ≥ 3.5) certifies ranking quality. We additionally report **MAE, Precision@10, Recall@10 and NDCG@10**. Reporting both families is the point: low error proves we *understand* the user; high MAP proves we *serve* them — and §8 shows they disagree.

06 Experimental Results

17,244-USER SUBSAMPLE · 2,000 EVALUATED USERS · CPU · SEED 42

MODEL	RMSE ↓	MAP@10 ↑	MAE ↓	PREC@10	REC@10	NDCG@10	FIT
Baseline (bias)	0.9399	**0.1843**•	0.7390	**0.1605**•	0.2969	**0.2899**•	0.1 s
Item-Item CF	0.9496	0.0854	0.7253	0.0912	0.1849	0.1532	18 s
Matrix Factorization	0.9118	0.1177	0.7110	0.1273	0.2287	0.2083	404 s
Ensemble (stacked)	**0.9053**•	0.1419	**0.7059**•	0.1401	0.2539	0.2384	74 s

• *marks the best value in each column. Ensemble blend weights — MF 0.675, Item-CF 0.317, Baseline 0.000.*

Reading the table. The Ensemble owns accuracy (RMSE 0.9053, MAE 0.7059). The Baseline owns ranking (MAP@10 0.1843, Precision 0.1605, NDCG 0.2899). Matrix Factorization is the strongest single accuracy model but the most expensive to train by three orders of magnitude. **Item-CF, tuned with significance shrinkage, improves on rating error but pays for it in ranking** — a deliberate accuracy-for-ranking trade we surface rather than hide. No "sophisticated" model beats the bias baseline at the one metric that reflects what users actually see.

07 Recommendation Examples

Top-10 for user 1730413 — explanation: *"because you rated The Sum of All Fears and Braveheart highly."*

01	Lost: Season 1 (2004)	3.88	06	My Favorite Wife (1940)	3.68
02	Firefly (2002)	3.81	07	All Creatures Great & Small: S1 (1978)	3.68
03	All Creatures Great & Small: S3 (1980)	3.75	08	The West Wing: Season 3 (2001)	3.67
04	Stargate SG-1: Season 7 (2003)	3.74	09	Farscape: The Peacekeeper Wars (2004)	3.67
05	CSI: Season 1 (2000)	3.71	10	Inspector Morse 15 (1990)	3.67

✓ SUCCESS

Inu-Yasha (2000)

User **1088135** · predicted **4.26** · actual **5**. The model surfaced a niche title the user genuinely loved.

✗ FAILURE

The Mummy (1999)

User **1940172** · predicted **4.60** · actual **2**. A popular crowd-pleaser overrode this user's specific taste — textbook popularity bias.

Every recommendation ships with a plain-language explanation grounded in the user's own top-rated titles — transparent by construction, not a black box.

08 Key Insights

> *Driving down squared error rewards cautious, near-mean predictions that rank the Top-10 poorly. Our stacker, optimising RMSE, set the baseline's weight to zero — discarding our best ranker to chase accuracy.*
>
> INSIGHT 01 — ACCURACY ≠ RANKING, DEMONSTRATED

— **RMSE-best ≠ MAP-best, proven.** The Ensemble wins RMSE (0.9053) yet the Baseline wins MAP@10 (0.1843, +30% over MF). Tune on the metric that matches the product goal — for discovery, that is ranking, not error.

— **Complexity rarely paid.** MF cost ~4,000× the baseline's training time for a 3% RMSE edge and worse ranking — a concrete case for "balance performance and simplicity."

— **One protocol caveat, disclosed.** MAP@10 ranks true items against *uniformly random* negatives, which are mostly obscure long-tail titles. This rewards popularity-aware models like the baseline; the accuracy-vs-ranking lesson holds, but the baseline's ranking edge is partly a property of the sampled-negative protocol.

— **Sparsity vs history.** Item-CF reaches its best on recall for users with rich, overlapping histories — a different operating point on the discovery curve.

09 Future Improvements

Train MF with a rank-aware objective (BPR / WARP) instead of squared error; scale with Alternating Least Squares and implicit-feedback signals; add Neural Collaborative Filtering for non-linear interactions; build a **hybrid** model folding in `movie_titles` metadata (year / decade) to attack **cold-start**; and add explicit coverage / diversity objectives so the system is rewarded for genuine discovery, not head-item accuracy.

10 Reproducibility

Every random operation is seeded (`--seed 42`); the exact command, git commit, and library versions are written to `results/run_config.json`. One command reproduces every number above:

```
python -m src.main --data-dir ../data --files combined_data_1.txt \
    --sample-users 20000 --models baseline itemcf mf --top-k 10 --relevance 3.5
```